



American International University- Bangladesh (AIUB)

Faculty of Engineering (EEE)

Course Name:	Microprocessor and Embedded Systems	Course Code:	COE3104
Semester:	Summer 2024-2025	Sec:	K
Lab Instructor:	Md. Ali Noor		

Experiment No:	06
Experiment Name:	Communication between two Arduino Boards using SPI

Group Members	Name	ID
1.	Basudeb Kundu	23-50856-1
2.	Prohlad Chandra Das	23-50922-1
3.	Debashis Kumar Das	23-50953-1
4.	Indronill Dutta Nill	23-50974-1
5.	Nafiur Rahman Nirob	23-50991-1

Performance Date:	30-8-25	Due Date:	03-09-25
--------------------------	---------	------------------	----------

Marking Rubrics (to be filled by Lab Instructor)

Category	Proficient [6]	Good [4]	Acceptable [2]	Unacceptable [1]	Secured Marks
Theoretical Background, Methods & procedures sections	All information, measures and variables are provided and explained.	All Information provided that is sufficient, but more explanation is needed.	Most information correct, but some information may be missing or inaccurate.	Much information missing and/or inaccurate.	
Results	All of the criteria are met; results are described clearly and accurately;	Most criteria are met, but there may be some lack of clarity and/or incorrect information.	Experimental results don't match exactly with the theoretical values and/or analysis is unclear.	Experimental results are missing or incorrect;	
Discussion	Demonstrates thorough and sophisticated understanding. Conclusions drawn are appropriate for analyses;	Hypotheses are clearly stated, but some concluding statements not supported by data or data not well integrated.	Some hypotheses missing or misstated; conclusions not supported by data.	Conclusions don't match hypotheses, not supported by data; no integration of data from different sources.	
General formatting	Title page, placement of figures and figure captions, and other formatting issues all correct.	Minor errors in formatting.	Major errors and/or missing information.	Not proper style in text.	
Writing & organization	Writing is strong and easy to understand; ideas are fully elaborated and connected; effective transitions between sentences; no typographic, spelling, or grammatical errors.	Writing is clear and easy to understand; ideas are connected; effective transitions between sentences; minor typographic, spelling, or grammatical errors.	Most of the required criteria are met, but some lack of clarity, typographic, spelling, or grammatical errors are present.	Very unclear, many errors.	
Comments:				Total Marks (Out of):	

Title: Communication between two Arduino Boards using SPI

Objectives

The objective of the "Communication between two Arduino Boards using SPI" experiment is to demonstrate the implementation of Serial Peripheral Interface (SPI) for inter-board communication in embedded systems. By connecting two Arduino boards via SPI, the experiment aims to establish a reliable and efficient data exchange mechanism. This involves configuring one Arduino as the master device and the other as the slave, enabling synchronous serial communication between them. Through this experiment, participants gain practical experience in utilizing SPI protocols for real-time data transfer, enabling applications such as sensor networks, control systems, and collaborative multi-device projects in the field of electronics and robotics.

The Serial Peripheral Interface (SPI) is a synchronous serial communication protocol originally developed by Motorola. It is widely used for short-distance communication, especially in embedded systems. SPI allows full-duplex data transmission, meaning data can be sent and received simultaneously between a master and one or more slave devices. The communication is carried out using four main lines: MISO (Master In Slave Out), MOSI (Master Out Slave In), SCK (Serial Clock), and SS (Slave Select). In an SPI setup, the master generates the clock signal and initiates data transfer, while the slave responds accordingly. The master controls which slave it communicates with by pulling the respective slave's SS line low.

In this experiment, two Arduino Uno boards are used—one configured as the master and the other as the slave. Each Arduino is connected to an LED and a push button. The objective is to use SPI communication to control the LED on the master side using the button on the slave side and vice versa. The Arduino SPI library is utilized to simplify the communication process. The master reads the state of its push button and sends the data to the slave, while simultaneously receiving the state of the slave's button. Based on the received data, each Arduino controls its LED, turning it on or off accordingly. Serial communication via the Arduino IDE is also used to monitor the data being sent and received. This setup effectively demonstrates the working of SPI communication between two microcontroller boards in a real-time embedded application.

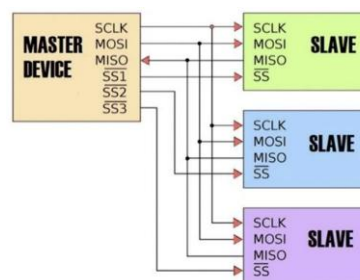


Fig1.1: Block diagram representation of SPI Master with several slaves.

Apparatus

1. Arduino IDE
2. Arduino UNO (2)
3. LED (2)
4. Push Button (2)
5. Resistors 10k, 2.2k (2+2)
6. Breadboard
7. Connecting Wires.
8. Circuit Diagram

Circuit Diagram:

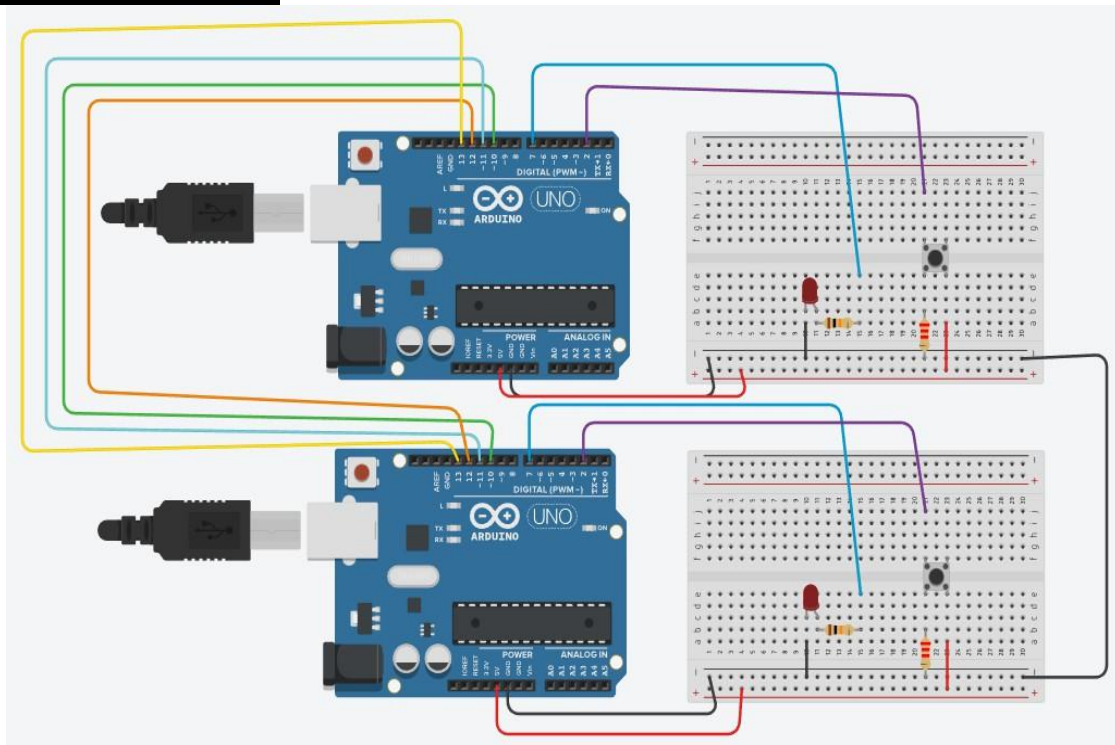


Fig1.2: Experimental setup of a LED light control using a switch from Master and slave

Code/Program

The following is the code for the implementation of a LED light control using a switch from Master and slave with the necessary code explanation:

MASTER Code (Arduino):

```
// SPI MASTER (ARDUINO)
#include <SPI.h>
```

```

#define LED 7
#define ipbutton 2

int buttonvalue;
int x;

void setup() {
  Serial.begin(115200); // Start Serial Communication
  pinMode(ipbutton, INPUT); // Set pin 2 as input (button)
  pinMode(LED, OUTPUT); // Set pin 7 as output (LED)
  pinMode(SS, OUTPUT); // Set SS as output to control Slave select

  SPI.begin(); // Begin SPI as Master
  SPI.setClockDivider(SPI_CLOCK_DIV8); // SPI Clock = 16MHz / 8 = 2MHz
  digitalWrite(SS, HIGH); // Disconnect Master from Slave initially
}

void loop() {
  byte Mastersend, Masterreceive;

  buttonvalue = digitalRead(ipbutton); // Read button state
  x = (buttonvalue == HIGH) ? 1 : 0; // Set value to send

  digitalWrite(SS, LOW); // Enable communication with Slave
  Mastersend = x;
  Masterreceive = SPI.transfer(Mastersend); // Send to Slave, receive from Slave
  digitalWrite(SS, HIGH); // Disable communication

  // Control LED based on data from Slave
  if (Masterreceive == 1) {
    digitalWrite(LED, HIGH);
    Serial.println("Master LED is ON");
  } else {
    digitalWrite(LED, LOW);
    Serial.println("Master LED is OFF");
  }

  delay(500); // Short delay for stability
}

```

SLAVE Code (Arduino):

```

// SPI SLAVE (ARDUINO)
#include <SPI.h>

#define LEDpin 7
#define buttonpin 2

volatile boolean received = false;

```

```

volatile byte Slaverceived = 0;
byte Slavesend = 0;
int buttonvalue;

void setup() {
  Serial.begin(115200);
  pinMode(buttonpin, INPUT); // Set pin 2 as input (button)
  pinMode(LEDpin, OUTPUT);   // Set pin 7 as output (LED)
  pinMode(MISO, OUTPUT);     // Set MISO as output to send data to Master

  SPCR |= _BV(SPE);          // Enable SPI in Slave Mode
  SPI.attachInterrupt();      // Enable SPI interrupt
}

ISR(SPI_STC_vect) {
  Slaverceived = SPDR;        // Receive data from Master
  received = true;            // Set flag
}

void loop() {
  if (received) {
    received = false; // Reset flag early

    // Act on received data
    if (Slaverceived == 1) {
      digitalWrite(LEDpin, HIGH);
      Serial.println("Slave LED is ON");
    } else {
      digitalWrite(LEDpin, LOW);
      Serial.println("Slave LED is OFF");
    }
    // Read button to send back to Master
    buttonvalue = digitalRead(buttonpin);
    Slavesend = (buttonvalue == HIGH) ? 1 : 0;

    SPDR = Slavesend; // Load data to send to Master
  }
  delay(10); // Small delay for stability
}

```

Experimental Circuits And Result

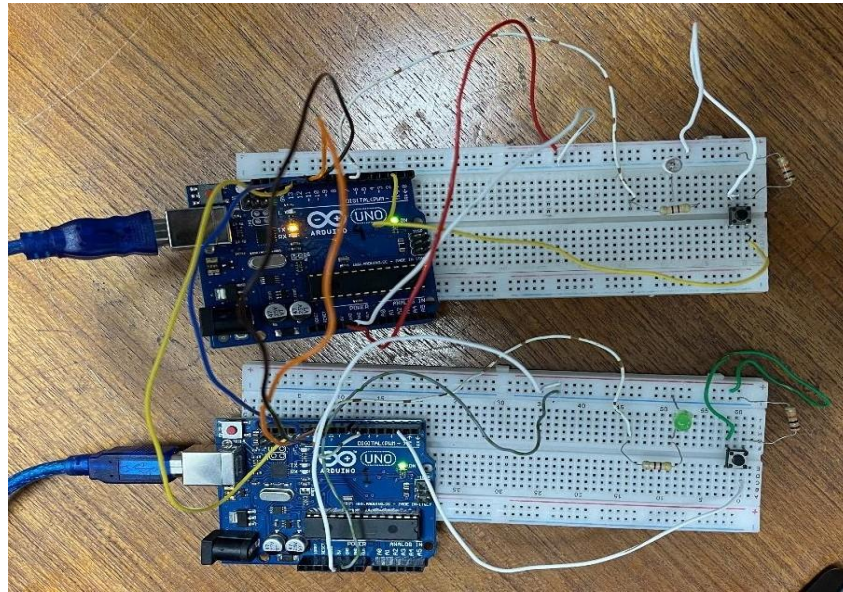


Fig2: Hardware implementation of a LED light control using a switch from Master and slave

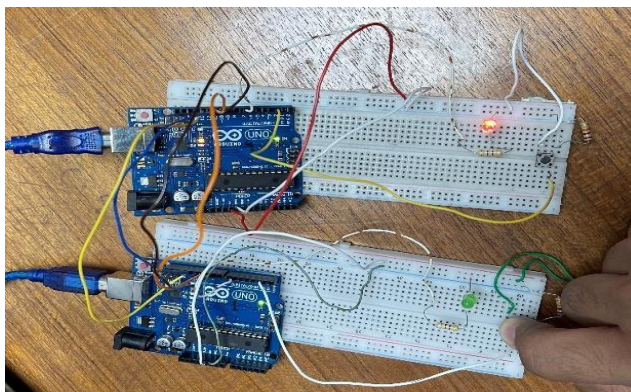


Fig3.1: Peripheral LED was ON in the SPI communication circuit when pressing the button of Controller

```
10:20:21.036 -> Slave LED is OFF
10:20:22.023 -> Slave LED is OFF
10:20:23.011 -> Slave LED is OFF
10:20:24.051 -> Slave LED is OFF
10:20:25.041 -> Slave LED is OFF
10:20:26.032 -> Slave LED is ON
10:20:27.020 -> Slave LED is ON
10:20:28.011 -> Slave LED is ON
```

Fig3.2: Serial Monitor of Peripheral when pressing Controller Data Button

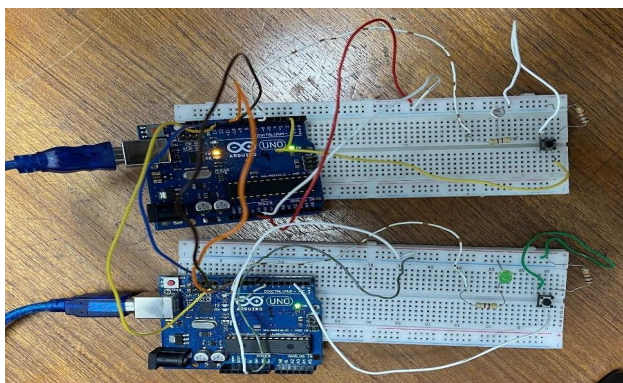


Fig4.1: Peripheral LED was OFF in the SPI communication circuit when not pressing the button of Controller

```
10:23:23.242 -> Slave LED is ON
10:23:24.278 -> Slave LED is ON
10:23:25.268 -> Slave LED is ON
10:23:26.256 -> Slave LED is ON
10:23:27.244 -> Slave LED is ON
10:23:28.281 -> Slave LED is ON
10:23:29.274 -> Slave LED is ON
10:23:30.264 -> Slave LED is OFF
10:23:31.256 -> Slave LED is OFF
10:23:32.246 -> Slave LED is OFF
10:23:33.282 -> Slave LED is OFF
10:23:34.266 -> Slave LED is OFF
10:23:35.255 -> Slave LED is OFF
```

Fig4.2: Serial Monitor of Peripheral when not pressing Controller Data Button

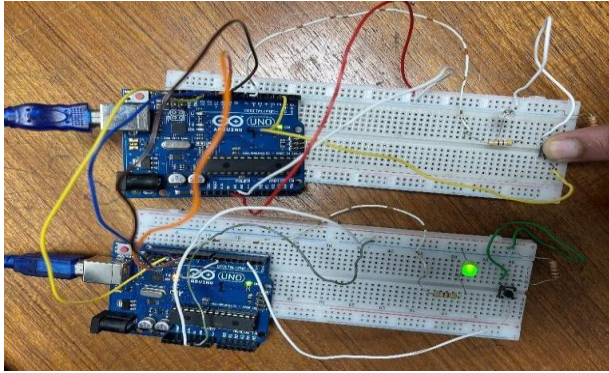


Fig5.1: The Controller LED is ON in the SPI communication circuit when pressing the button of Peripheral

```
10:21:32.068 -> Master LED is OFF
10:21:33.060 -> Master LED is OFF
10:21:34.098 -> Master LED is OFF
10:21:35.090 -> Master LED is OFF
10:21:36.085 -> Master LED is OFF
10:21:37.079 -> Master LED is OFF
10:21:38.075 -> Master LED is OFF
10:21:39.069 -> Master LED is ON
10:21:40.109 -> Master LED is ON
10:21:41.108 -> Master LED is ON
10:21:42.103 -> Master LED is ON
10:21:43.099 -> Master LED is ON
```

Fig5.2: Serial Monitor of Peripheral when pressing Peripheral Data Button

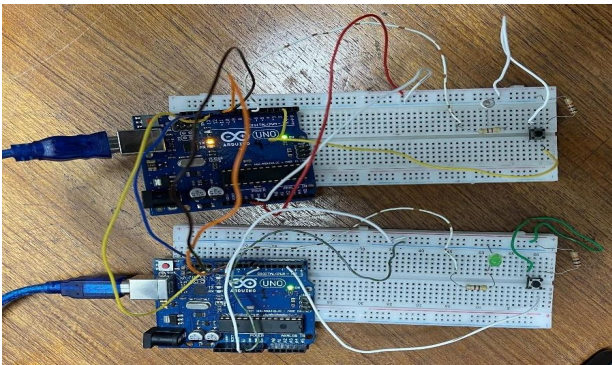


Fig6.1: Controller LED was OFF in the SPI communication circuit when not pressing the button of Peripheral

```
10:22:39.166 -> Master LED is ON
10:22:40.202 -> Master LED is ON
10:22:41.193 -> Master LED is ON
10:22:42.181 -> Master LED is ON
10:22:43.215 -> Master LED is ON
10:22:44.203 -> Master LED is ON
10:22:45.193 -> Master LED is OFF
10:22:46.181 -> Master LED is OFF
10:22:47.215 -> Master LED is OFF
10:22:48.199 -> Master LED is OFF
10:22:49.185 -> Master LED is OFF
```

Fig6.2: Serial Monitor of Peripheral when not pressing Peripheral Data Button

Explanation:

In this experiment, there were two separate experimental outcomes. The first result was obtained when pressing the data transfer button on the Controller. Upon pressing the button, after a few seconds, the LED on the Peripheral lit up. This indicates that the signal to turn on the LED was successfully transmitted from the Controller to the Peripheral via the SPI communication interface. The Serial Monitor of the Controller also displayed the message: "Slave LED is ON." When the button on the Controller was released, the LED connected to the Peripheral turned off, confirming that the signal to turn off the LED was also transmitted correctly, with the Serial Monitor showing "Slave LED is OFF."

The second result was obtained when pressing the data transfer button on the Peripheral. Upon pressing the button, after a few seconds, the LED on the Controller lit up, indicating that the data to turn on the Controller's LED was sent from the Peripheral via SPI. The Serial Monitor of the Peripheral showed "Master LED is ON." When the button was released, the LED on the Controller turned off, and the monitor displayed "Master LED is OFF," confirming successful bidirectional communication.

Simulation Circuits And Result

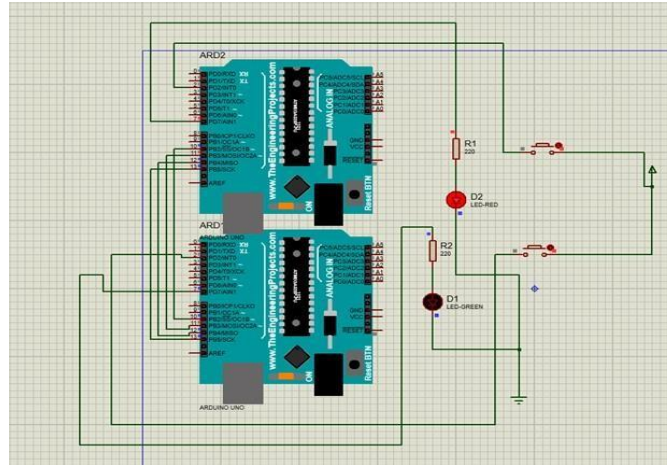


Fig7: Peripheral LED was ON in the SPI communication circuit simulation when pressing the button of Controller

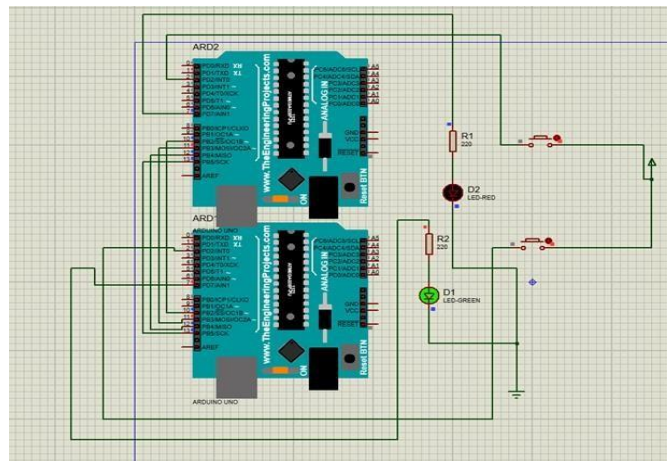


Fig8: Controller LED was ON in the SPI communication circuit simulation when pressing the button of Peripheral

Explanation:

In this simulation, two Arduino Uno boards, resistors, GREEN and RED LEDs, and push buttons were configured according to the wiring instructions provided in the commented Arduino code. The buttons used to transmit data from one microcontroller to the other were properly connected based on the circuit diagram and code logic. Both Arduino boards were linked using the SPI communication interface via pins 10 to 13.

The program for SPI communication was written and verified using the Arduino IDE. Upon successful verification, a HEX file was generated. This HEX file was then loaded into the Arduino modules in the Proteus simulation environment to control their behavior according to the programmed logic.

Once the simulation was run, the LEDs responded to the corresponding button presses, reflecting successful bidirectional SPI communication. The results observed in the simulation closely matched the outcomes of the hardware experiment, validating the correctness of both the circuit design and the SPI protocol implementation.

Discussion

This experiment successfully demonstrated the practical application of SPI (Serial Peripheral Interface) communication between two Arduino Uno boards. By connecting each board with an LED and a push button, we were able to verify the bidirectional data exchange using SPI, where each Arduino could send and receive signals simultaneously. The use of the `SPI.transfer()` function on the Controller and the SPI interrupt on the Peripheral allowed real-time control of LEDs based on the button inputs from the opposite board.

The serial monitor readings provided further confirmation of successful communication. Messages such as "Master LED is ON" or "Slave LED is OFF" indicated that the data being transferred was correctly received and interpreted. The synchronization between hardware behavior and serial output confirmed that the SPI protocol was implemented and functioning properly. The accurate reflection of button states through the LEDs on both sides showed that SPI can be reliably used for fast, real-time communication in embedded systems.

Additionally, the simulation conducted using Proteus produced results consistent with the physical hardware implementation. This helped validate the code logic and circuit connections before actual deployment. Proper wiring of the SPI pins (MISO, MOSI, SCK, and SS) and careful setup of pull-up or pull-down resistors were crucial for stable communication. Overall, the experiment deepened our understanding of synchronous serial communication and highlighted SPI's effectiveness in simple but meaningful microcontroller-to-microcontroller interactions.

Reference

- [1] "SPI Communication Between Two Arduino Boards," *Circuit Digest*. [Online]. Available: <https://circuitdigest.com/microcontroller-projects/spi-communication-between-two-arduino-boards> [Accessed: May 5, 2025].
- [2] Arduino, "SPI - Serial Peripheral Interface - Arduino Reference," *Arduino Documentation*. [Online]. Available: <https://www.arduino.cc/en/Reference/SPI> [Accessed: May 5, 2025].
- [3] Proteus Design Suite, "How to Simulate Arduino Projects in Proteus," *Labcenter Electronics*. [Online]. Available: <https://www.labcenter.com> [Accessed: May 5, 2025].
- [4] M. Ali Noor, *Lab Manual: Microprocessor and Embedded Systems Laboratory*, American International University-Bangladesh (AIUB), Spring 2024-25.